CHƯƠNG 7

KIẾN TRÚC PHẦN MỀM

(Software Architecture)

Học phần: Phân tích và Thiết kế Hệ thống thông tin hướng đối tượng

Ngành: Hệ thống thông tin quản lý

Giới thiệu chương

Chương này trình bày các khái niệm nền tảng về **kiến trúc phần mềm**, đi từ các mô hình kiến trúc truyền thống như **Client-Server** và **kiến trúc phân tầng**, qua các kiến trúc chuyển tiếp như **hướng dịch vụ (SOA)** và **hướng sự kiện (Event-Driven)**, đến các kiến trúc hiện đại đang được doanh nghiệp ứng dụng rộng rãi như **vi dịch vụ (Microservices)**, **thuần mây (Cloud-Native)** và **phi máy chủ (Serverless)**. Thông qua chương này, sinh viên có khả năng phân biệt kiến trúc với thiết kế chi tiết, nhận diện đặc điểm và đánh giá ưu nhược điểm của từng mô hình kiến trúc, đồng thời vận dụng được các thuộc tính chất lượng phi chức năng để lựa chọn kiến trúc phù hợp cho một bài toán nghiệp vụ cụ thể. Đây là kiến thức nền tảng giúp sinh viên chuyển từ tư duy phân tích thiết kế ở mức chi tiết — lớp, đối tượng, phương thức — sang tư duy tổ chức hệ thống ở mức tổng thể, phục vụ trực tiếp cho việc ra quyết định công nghệ trong các dự án hệ thống thông tin doanh nghiệp sau này.

Mục tiêu chương

- Phân biệt được **kiến trúc phần mềm (Software Architecture)** với **thiết kế phần mềm chi tiết (Software Design)**.
- Trình bày được các thành phần cốt lõi cấu thành nên một kiến trúc hệ thống.
- Mô tả đặc điểm, phân tích ưu và nhược điểm của các mô hình kiến trúc truyền thống, chuyển tiếp và hiện đại.
- Giải thích được các thuộc tính chất lượng phi chức năng (khả năng mở rộng, độ tin cậy, bảo mật, hiệu năng) liên quan đến kiến trúc.
- Vận dụng kiến thức để đánh giá, lựa chọn kiến trúc phù hợp cho một bài toán thực tế và tài liệu hóa bằng mô hình C4.

7.1. Tổng quan về Kiến trúc Phần mềm

Trong các chương trước, sinh viên đã làm quen với việc phân tích và thiết kế hệ thống ở mức **chi tiết**: xây dựng use case, mô hình hóa lớp bằng class diagram, mô tả luồng tương tác giữa các đối tượng bằng sequence diagram. Những mô hình đó trả lời câu hỏi "hệ thống gồm những lớp, đối tượng nào và chúng tương tác ra sao". Phần này giúp sinh viên bước lên một mức trừu tượng cao hơn: hiểu **kiến trúc tổng thể** của hệ thống — bức tranh lớn quyết định hệ thống được tổ chức, triển khai và vận hành như thế nào.

7.1.1. Khái niệm Kiến trúc phần mềm

Kiến trúc phần mềm (Software Architecture) là cấu trúc tổng thể của một hệ thống, bao gồm các thành phần chính, mối quan hệ giữa chúng, và các nguyên tắc chi phối việc thiết kế, vận hành hệ thống đó. Nói cách khác, kiến trúc trả lời câu hỏi: hệ thống được **chia thành những khối lớn nào**, các khối đó **đặt ở đâu** (trên máy chủ nào, trung tâm dữ liệu nào) và **giao tiếp với nhau như thế nào**.

Thiết kế phần mềm (Software Design), ngược lại, tập trung vào cách hiện thực từng khối đó ở mức chi tiết: định nghĩa lớp, thuộc tính, phương thức, thuật toán, mẫu thiết kế (design pattern) bên trong một thành phần. Có thể hình dung: nếu kiến trúc là bản vẽ quy hoạch tổng thể của một khu đô thị — gồm

những khu chức năng nào, đường giao thông kết nối ra sao — thì thiết kế chi tiết là bản vẽ kết cấu bên trong từng tòa nhà.

Tiêu chí	Kiến trúc phần mềm (Architecture)	Thiết kế chi tiết (Design)
Mức trừu tượng	Toàn hệ thống, các khối lớn	Từng module, lớp, phương thức
Câu hỏi trả lời	Hệ thống gồm những thành phần nào, đặt ở đâu, giao tiếp ra sao	Bên trong một thành phần được lập trình như thế nào
Chi phí thay đổi	Rất cao, ảnh hưởng toàn hệ thống	Thấp hơn, khoanh vùng trong module
Người quyết định	Kiến trúc sư hệ thống, quản lý dự án cấp cao	Lập trình viên, trưởng nhóm kỹ thuật

7.1.2. Mối liên hệ với Phân tích và Thiết kế Hướng đối tượng (OOA&D)

Các mô hình OOA&D mà sinh viên đã học không tách rời khỏi kiến trúc — chúng chính là nguyên liệu đầu vào để xây dựng kiến trúc. Cụ thể:

- **Use case** giúp xác định các chức năng chính hệ thống phải cung cấp, từ đó ước lượng quy mô, mức độ phức tạp — làm cơ sở chọn kiến trúc phù hợp (một use case đơn giản có thể chỉ cần kiến trúc nguyên khối, trong khi hệ thống với hàng chục use case phức tạp, nhiều bên liên quan có thể cần vi dịch vụ).
- **Class diagram**, đặc biệt là việc phân loại **lớp biên (boundary), lớp điều khiển (control) và lớp thực thể (entity)**, ánh xạ trực tiếp vào các tầng kiến trúc: lớp biên tương ứng tầng trình diễn, lớp điều khiển tương ứng tầng xử lý nghiệp vụ, lớp thực thể tương ứng tầng dữ liệu.
- **Sequence diagram** mô tả luồng gọi giữa các đối tượng; khi các đối tượng đó được đặt trên các máy chủ khác nhau, sequence diagram chính là cơ sở để vẽ **deployment diagram** — biểu đồ triển khai thể hiện các node vật lý và giao tiếp mạng giữa chúng.

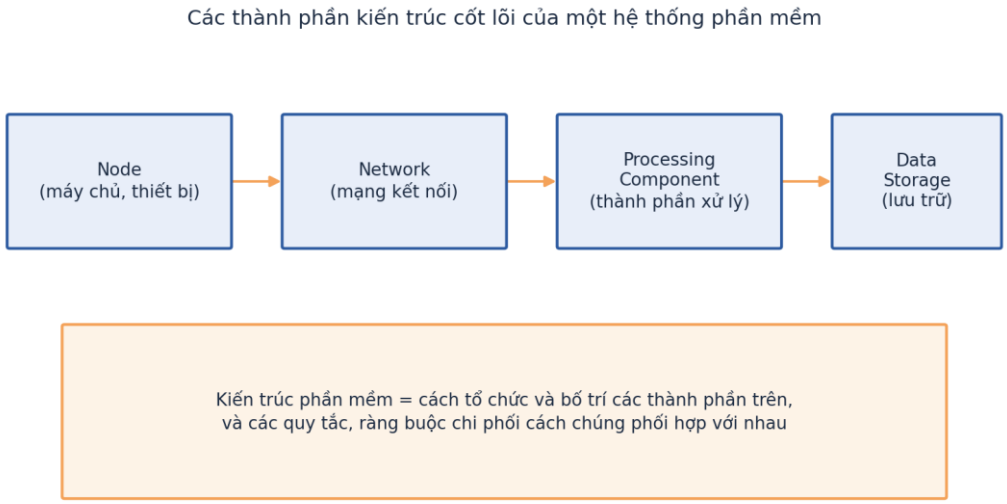
Nói ngắn gọn: OOA&D thiết kế "bộ khung logic" của hệ thống; kiến trúc quyết định "bộ khung logic" đó được đóng gói, phân bố và vận hành trên hạ tầng thực tế như thế nào.

7.1.3. Các thành phần kiến trúc cốt lõi

Dù kiến trúc được tổ chức theo mô hình nào, hầu hết hệ thống đều được xây dựng từ bốn loại thành phần cốt lõi sau:

- **Node**: các máy chủ vật lý hoặc ảo, thiết bị đầu cuối (máy tính, điện thoại) nơi phần mềm được cài đặt và chạy.
- **Network (mạng)**: hạ tầng kết nối các node với nhau, quyết định tốc độ và độ tin cậy của việc trao đổi dữ liệu.
- **Processing Component (thành phần xử lý)**: các khối chương trình thực hiện logic nghiệp vụ — ví dụ dịch vụ tính điểm tín dụng, dịch vụ xác thực người dùng.

- **Data Storage (lưu trữ dữ liệu)**: nơi lưu trữ dữ liệu bền vững — cơ sở dữ liệu quan hệ, kho dữ liệu NoSQL, hệ thống file.



Hình 7.1 — Bốn thành phần kiến trúc cốt lõi và mối quan hệ giữa chúng

Sự khác biệt giữa các kiểu kiến trúc trình bày trong chương này, thực chất, nằm ở **cách bố trí và kết nối bốn loại thành phần trên** — cùng những nguyên tắc chi phối việc chúng phối hợp với nhau.

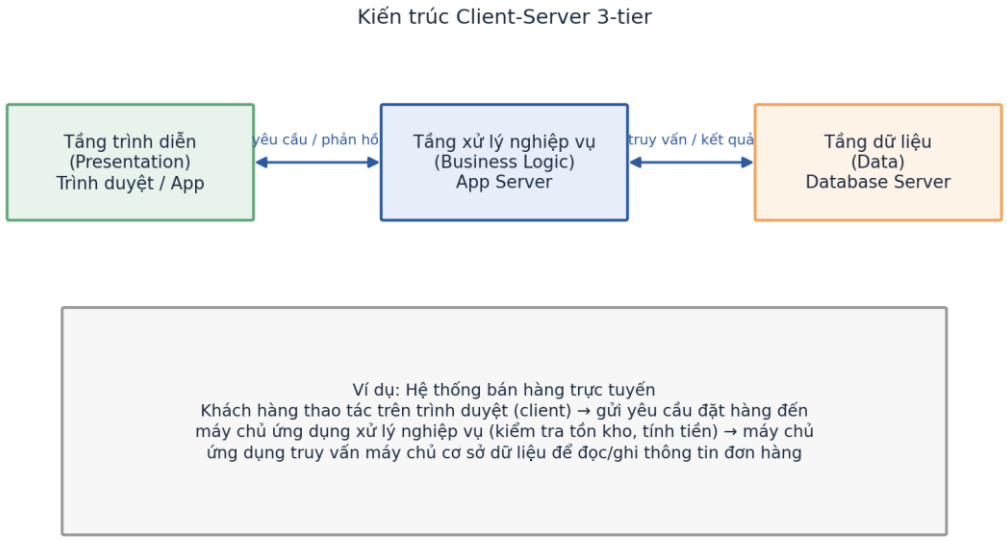
7.2. Các Mô hình Kiến trúc Truyền thống

Đây là những kiến trúc nền tảng, xuất hiện sớm nhất và vẫn còn được sử dụng phổ biến, đặc biệt trong các hệ thống thông tin doanh nghiệp quy mô vừa và nhỏ. Sinh viên cần nắm vững nhóm này trước khi tiếp cận các kiến trúc phân tán phức tạp hơn ở phần sau.

7.2.1. Kiến trúc Client-Server (Khách - Chủ)

Kiến trúc Client-Server tổ chức hệ thống thành hai vai trò: **client** (bên gửi yêu cầu, thường là giao diện người dùng) và **server** (bên xử lý yêu cầu và trả kết quả). Tùy theo số lớp được tách ra, ta có các biến thể:

- **Mô hình 2-tier**: client giao tiếp trực tiếp với server cơ sở dữ liệu, logic nghiệp vụ thường được nhúng ngay trong ứng dụng client. Đơn giản, triển khai nhanh, nhưng khó bảo trì khi nghiệp vụ thay đổi vì phải cập nhật lại từng máy client.
- **Mô hình 3-tier**: tách riêng tầng trình diễn (presentation), tầng xử lý nghiệp vụ (business logic) và tầng dữ liệu (data), mỗi tầng có thể đặt trên máy chủ riêng. Đây là mô hình phổ biến nhất hiện nay cho các ứng dụng web doanh nghiệp.
- **Mô hình N-tier**: mở rộng 3-tier bằng cách tách thêm các tầng trung gian khác (ví dụ tầng tích hợp, tầng cache) khi hệ thống phức tạp hơn.



Hình 7.2 — Kiến trúc Client-Server 3-tier

	Ưu điểm	Nhược điểm
Client-Server	Tách bạch trách nhiệm giữa các tầng, dễ bảo trì và nâng cấp từng tầng độc lập; tận dụng tài nguyên máy chủ thay vì phụ thuộc hoàn toàn vào máy client	Máy chủ trở thành điểm chịu tải chính, cần đầu tư hạ tầng; độ trễ mạng ảnh hưởng trải nghiệm nếu thiết kế không tối ưu

7.2.2. Kiến trúc Nguyên khối (Monolithic Architecture)

Kiến trúc nguyên khối đóng gói toàn bộ chức năng của hệ thống — giao diện, xử lý nghiệp vụ, truy xuất dữ liệu — thành **một khối mã nguồn duy nhất**, biên dịch và triển khai (deploy) cùng một lần. Đây thường là lựa chọn mặc định khi bắt đầu một dự án mới vì đơn giản, dễ phát triển và dễ kiểm thử trong giai đoạn đầu.

Tuy nhiên, khi hệ thống phát triển lớn dần — ví dụ một hệ thống quản lý bán hàng ban đầu chỉ có vài chức năng, sau vài năm bổ sung thêm module kho, module khuyến mãi, module báo cáo mà không tái cấu trúc — mã nguồn dễ trở thành một khối rối rắm, khó kiểm soát, thường được gọi là **"Big Ball of Mud"**. Khi đó, việc mở rộng quy mô (scale) hay chỉ sửa một chức năng nhỏ cũng buộc phải build và triển khai lại toàn bộ hệ thống.

7.2.3. Kiến trúc Phân tầng (Layered/Tiered Architecture)

Kiến trúc phân tầng tổ chức mã nguồn theo các tầng logic xếp chồng lên nhau, mỗi tầng chỉ giao tiếp với tầng liền kề, phổ biến nhất là ba tầng: **Presentation** (giao diện), **Business Logic** (xử lý nghiệp vụ) và **Data Access** (truy xuất dữ liệu). Khác với client-server vốn phân chia theo **vị trí vật lý** (máy nào), phân tầng phân chia theo **trách nhiệm logic** trong mã nguồn, dù các tầng này vẫn có thể được triển khai trên cùng

một máy chủ (như trong kiến trúc nguyên khối) hoặc trên nhiều máy chủ khác nhau (như trong client-server 3-tier).

Một cách áp dụng phổ biến của kiến trúc phân tầng là mẫu **MVC (Model-View-Controller)**: **Model** quản lý dữ liệu và quy tắc nghiệp vụ, **View** hiển thị giao diện, **Controller** tiếp nhận thao tác người dùng và điều phối giữa Model và View. MVC giúp thay đổi giao diện mà không ảnh hưởng logic nghiệp vụ, và ngược lại.



Hình 7.3 — So sánh kiến trúc nguyên khối và kiến trúc phân tầng theo mẫu MVC

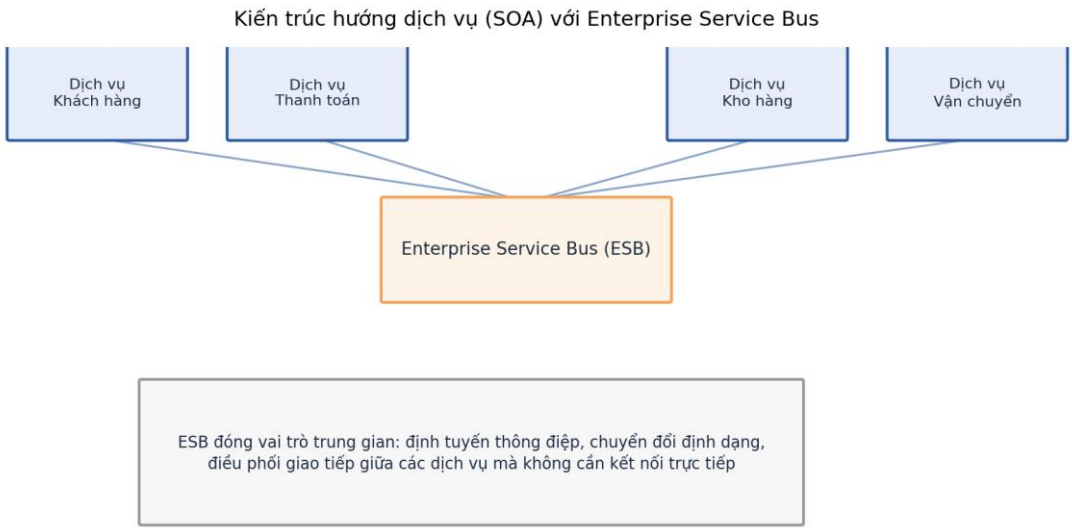
Ví dụ minh họa: Hệ thống quản lý thư viện của một trường đại học được xây dựng theo kiến trúc nguyên khối, phân tầng theo MVC: sinh viên tra cứu và mượn sách qua giao diện web (View); Controller tiếp nhận yêu cầu mượn sách, gọi Model kiểm tra số lượng sách còn lại và cập nhật trạng thái mượn — toàn bộ được đóng gói và triển khai như một ứng dụng duy nhất.

7.3. Kiến trúc Hướng Dịch vụ và Sự kiện

Khi các hệ thống thông tin doanh nghiệp cần kết nối với nhiều hệ thống khác nhau — ví dụ hệ thống bán hàng cần liên kết với hệ thống kế toán, hệ thống quản lý kho của đối tác — kiến trúc nguyên khối và client-server truyền thống bộc lộ giới hạn. Nhóm kiến trúc này giúp sinh viên chuyển từ tư duy hệ thống "đóng", tự chứa, sang tư duy hệ thống "mở", nhiều thành phần độc lập cùng phối hợp.

7.3.1. Kiến trúc Hướng Dịch vụ (SOA — Service-Oriented Architecture)

Kiến trúc hướng dịch vụ (SOA) tổ chức hệ thống thành các **dịch vụ (service)** độc lập, mỗi dịch vụ cung cấp một chức năng nghiệp vụ cụ thể (ví dụ dịch vụ thanh toán, dịch vụ tra cứu khách hàng) và giao tiếp với nhau qua các giao thức chuẩn. Thành phần trung tâm của SOA là **Enterprise Service Bus (ESB)** — một tầng trung gian đóng vai trò định tuyến thông điệp, chuyển đổi định dạng dữ liệu và điều phối giao tiếp giữa các dịch vụ, giúp các dịch vụ không cần biết chi tiết kỹ thuật của nhau.



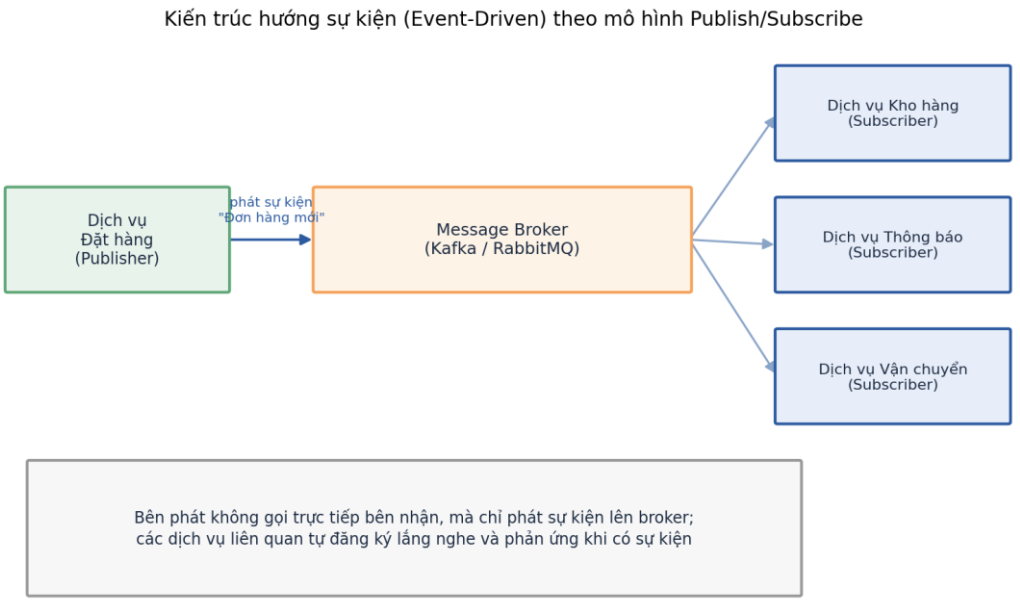
Hình 7.4 — Kiến trúc SOA với Enterprise Service Bus (ESB) làm trung gian kết nối các dịch vụ

- **Ưu điểm**: tái sử dụng dịch vụ giữa nhiều ứng dụng; tích hợp được các hệ thống cũ (legacy) với hệ thống mới thông qua ESB.
- **Nhược điểm**: ESB dễ trở thành **điểm nghẽn (bottleneck)** và **điểm lỗi duy nhất (single point of failure)** khi số lượng dịch vụ tăng lên; cấu hình và vận hành ESB phức tạp, chi phí bản quyền phần mềm ESB thương mại cao.

Vì những hạn chế trên, nhiều doanh nghiệp hiện nay chuyển dần từ SOA có ESB tập trung sang **kiến trúc vi dịch vụ** (trình bày ở phần 7.4), nơi việc giao tiếp giữa các dịch vụ nhẹ và phi tập trung hơn.

7.3.2. Kiến trúc Hướng Sự kiện (Event-Driven Architecture)

Kiến trúc hướng sự kiện tổ chức hệ thống xoay quanh việc phát sinh, phát hiện và phản ứng với các **sự kiện (event)** — ví dụ sự kiện "đơn hàng mới được tạo", "thanh toán thành công". Thay vì một thành phần gọi trực tiếp hàm của thành phần khác, bên phát sự kiện (**publisher**) chỉ đưa sự kiện lên một **message broker** (như **Kafka**, **RabbitMQ**); các thành phần quan tâm (**subscriber**) tự đăng ký lắng nghe và xử lý khi sự kiện xảy ra — mô hình này gọi là **Publish/Subscribe (Pub/Sub)**.



Hình 7.5 — Kiến trúc hướng sự kiện theo mô hình Publish/Subscribe

Ví dụ minh họa: Trong một sàn thương mại điện tử, khi khách hàng đặt hàng thành công, dịch vụ Đặt hàng chỉ cần phát sự kiện "Đơn hàng mới" lên message broker. Dịch vụ Kho hàng lắng nghe để trừ tồn kho, dịch vụ Thông báo lắng nghe để gửi email xác nhận, dịch vụ Vận chuyển lắng nghe để tạo phiếu giao hàng — cả ba xử lý song song, độc lập, mà dịch vụ Đặt hàng không cần biết chúng tồn tại.

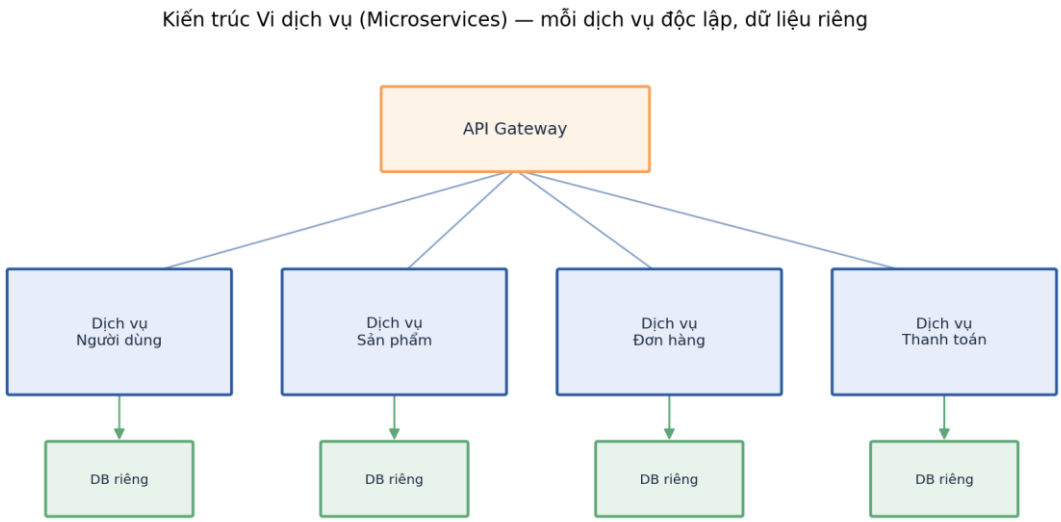
Ưu điểm nổi bật của kiến trúc hướng sự kiện là **giảm sự phụ thuộc trực tiếp (loose coupling)** giữa các thành phần và dễ dàng bổ sung thành phần lắng nghe mới mà không sửa thành phần phát sự kiện. Đánh đổi là hệ thống khó theo dõi luồng xử lý tổng thể hơn (nhiều thành phần xử lý bất đồng bộ, song song) và cần cơ chế xử lý khi một sự kiện được xử lý trùng lặp hoặc bị mất.

7.4. Các Kiến trúc Phần mềm Hiện đại

Nhóm kiến trúc này phản ánh xu hướng công nghệ mà các doanh nghiệp lớn — đặc biệt là các công ty công nghệ tài chính, thương mại điện tử — đang áp dụng để đạt khả năng mở rộng và tốc độ phát triển sản phẩm cao.

7.4.1. Kiến trúc Vi dịch vụ (Microservices Architecture)

Kiến trúc vi dịch vụ chia một hệ thống nguyên khối lớn thành nhiều **dịch vụ nhỏ, độc lập**, mỗi dịch vụ đảm nhiệm một chức năng nghiệp vụ hẹp (ví dụ dịch vụ Người dùng, dịch vụ Sản phẩm, dịch vụ Đơn hàng), có thể được phát triển, kiểm thử, triển khai và mở rộng **hoàn toàn độc lập** với các dịch vụ khác. Mỗi dịch vụ thường sở hữu **cơ sở dữ liệu riêng**, tránh việc nhiều dịch vụ cùng truy cập trực tiếp một cơ sở dữ liệu chung.



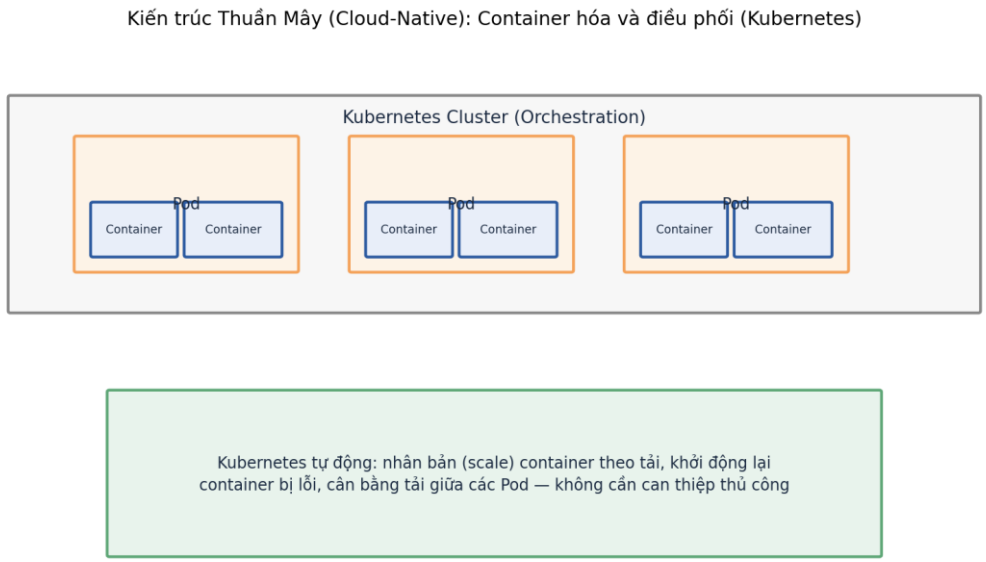
Hình 7.6 — Kiến trúc Vi dịch vụ: mỗi dịch vụ độc lập, có dữ liệu riêng, giao tiếp qua API Gateway

- **Ưu điểm**: mở rộng độc lập từng dịch vụ theo đúng nhu cầu tải (ví dụ chỉ cần nhân bản dịch vụ Thanh toán vào dịp cao điểm); nhóm phát triển có thể triển khai (deploy) dịch vụ của mình mà không ảnh hưởng các nhóm khác; dễ áp dụng công nghệ khác nhau cho từng dịch vụ.
- **Thách thức**: quản lý một hệ thống phân tán với hàng chục, hàng trăm dịch vụ phức tạp hơn nhiều so với một khối duy nhất (giám sát, gỡ lỗi, kiểm thử tích hợp); đảm bảo **tính nhất quán dữ liệu** giữa các cơ sở dữ liệu riêng biệt là bài toán khó, thường phải đánh đổi giữa nhất quán tức thời và nhất quán cuối cùng (eventual consistency).

7.4.2. Kiến trúc Thuần Mây (Cloud-Native Architecture)

Kiến trúc thuần mây thiết kế ứng dụng để tận dụng tối đa đặc tính của hạ tầng điện toán đám mây (cloud computing) ngay từ đầu, thay vì chỉ đơn thuần đưa một ứng dụng cũ lên cloud. Hai công nghệ nền tảng của kiến trúc thuần mây là:

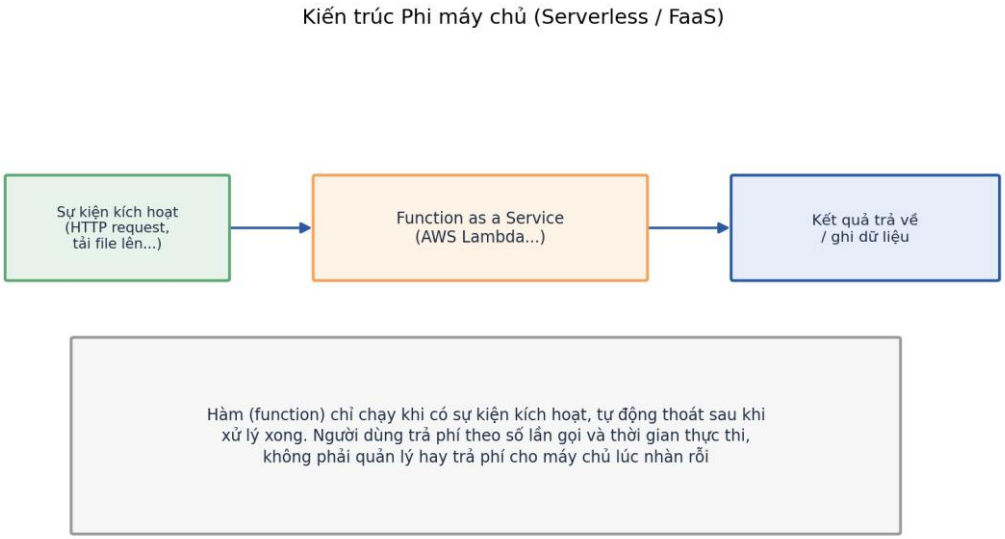
- **Containerization (đóng gói container)**: đóng gói ứng dụng cùng toàn bộ môi trường chạy (thư viện, biến cấu hình) vào một **container** — phổ biến nhất là công cụ **Docker** — đảm bảo ứng dụng chạy nhất quán trên mọi môi trường, từ máy lập trình viên đến máy chủ production.
- **Orchestration (điều phối container)**: khi số lượng container lớn, cần công cụ điều phối như **Kubernetes** để tự động triển khai, nhân bản (scale) khi tải tăng, khởi động lại container bị lỗi và cân bằng tải giữa các container.



Hình 7.7 — Container hóa bằng Docker và điều phối bằng Kubernetes

7.4.3. Kiến trúc Phi máy chủ (Serverless Architecture)

Kiến trúc phi máy chủ không có nghĩa là không có máy chủ, mà nghĩa là đội ngũ phát triển **không cần quản lý** máy chủ — nhà cung cấp dịch vụ cloud tự động cấp phát, vận hành và mở rộng hạ tầng. Mô hình phổ biến nhất là **FaaS (Function as a Service)**, ví dụ **AWS Lambda**: lập trình viên chỉ viết một hàm (function) xử lý một tác vụ cụ thể; hàm này chỉ được khởi chạy khi có sự kiện kích hoạt (ví dụ có yêu cầu HTTP gửi đến, có file mới được tải lên) và tự động dừng sau khi xử lý xong.



Hình 7.8 — Kiến trúc phi máy chủ theo mô hình FaaS

Lợi ích lớn nhất của serverless là mô hình **trả tiền theo mức sử dụng thực tế** (pay-as-you-go) — không phát sinh chi phí khi không có yêu cầu nào được xử lý — và giảm gánh nặng vận hành hạ tầng cho đội

ngũ kỹ thuật. Đánh đổi là ràng buộc chặt vào nhà cung cấp cloud cụ thể (vendor lock-in) và không phù hợp cho các tác vụ cần chạy liên tục, thời gian dài.

7.5. Yêu cầu Phi chức năng và Thuộc tính Chất lượng

Một kiến trúc tốt không chỉ giúp hệ thống **chạy được** đúng chức năng nghiệp vụ, mà còn phải **chạy tốt**: đáp ứng được lượng người dùng lớn, ổn định, an toàn và phản hồi nhanh. Đây chính là các **yêu cầu phi chức năng (non-functional requirement)**, khác với yêu cầu chức năng vốn mô tả "hệ thống làm gì".

7.5.1. Khả năng mở rộng (Scalability)

Khả năng mở rộng là khả năng hệ thống đáp ứng khi lượng người dùng hoặc khối lượng dữ liệu tăng lên, theo hai hướng:

- **Scale-up (mở rộng theo chiều dọc)**: nâng cấp cấu hình của một máy chủ hiện có — thêm CPU, RAM. Đơn giản nhưng có giới hạn vật lý và tạo điểm lỗi duy nhất.
- **Scale-out (mở rộng theo chiều ngang)**: thêm nhiều máy chủ chạy song song để cùng gánh tải. Phức tạp hơn khi triển khai (cần cân bằng tải) nhưng gần như không giới hạn khả năng mở rộng — đây là hướng tiếp cận phù hợp với kiến trúc vi dịch vụ và cloud-native.

7.5.2. Độ tin cậy và Tính sẵn sàng cao (Reliability & High Availability)

Độ tin cậy là khả năng hệ thống hoạt động đúng, ổn định trong thời gian dài mà không xảy ra lỗi. **Tính sẵn sàng cao** là khả năng hệ thống luôn sẵn sàng phục vụ, giảm thiểu thời gian ngừng hoạt động (downtime), thường được đo bằng tỷ lệ phần trăm thời gian hoạt động (ví dụ "99.9% uptime"). Kỹ thuật phổ biến để đạt được hai thuộc tính này là **dự phòng (redundancy)**: duy trì nhiều bản sao của cùng một thành phần (nhiều máy chủ, nhiều bản sao cơ sở dữ liệu), khi một bản sao gặp sự cố, các bản sao còn lại tiếp tục phục vụ mà người dùng không nhận thấy gián đoạn.

7.5.3. Bảo mật kiến trúc (Security)

Kiến trúc cần được thiết kế để bảo vệ hệ thống khỏi truy cập và tấn công trái phép, đặc biệt quan trọng với hệ thống thông tin có dữ liệu nhạy cảm như tài chính, ngân hàng. Các cơ chế bảo mật thường được tích hợp ở mức kiến trúc bao gồm:

- **Tường lửa (firewall)**: kiểm soát và lọc lưu lượng mạng ra/vào hệ thống.
- **Mã hóa (encryption)**: bảo vệ dữ liệu khi truyền trên mạng và khi lưu trữ.
- **Quản lý xác thực và phân quyền (authentication & authorization)**: xác minh danh tính người dùng và giới hạn quyền truy cập theo vai trò — đặc biệt phức tạp trong hệ thống phân tán nhiều dịch vụ, nơi mỗi lời gọi giữa các dịch vụ cũng cần được xác thực.

7.5.4. Hiệu năng (Performance)

Hiệu năng phản ánh tốc độ phản hồi và khả năng xử lý của hệ thống. Hai kỹ thuật kiến trúc phổ biến để cải thiện hiệu năng:

- **Caching (bộ nhớ đệm)**: lưu tạm kết quả của các thao tác tốn thời gian (ví dụ kết quả truy vấn cơ sở dữ liệu được truy cập thường xuyên) để phục vụ nhanh cho các yêu cầu sau, không phải tính toán lại từ đầu.
- **Load balancing (cân bằng tải)**: phân phối đều các yêu cầu đến giữa nhiều máy chủ hoặc nhiều bản sao dịch vụ, tránh tình trạng một máy chủ quá tải trong khi máy chủ khác nhàn rỗi.

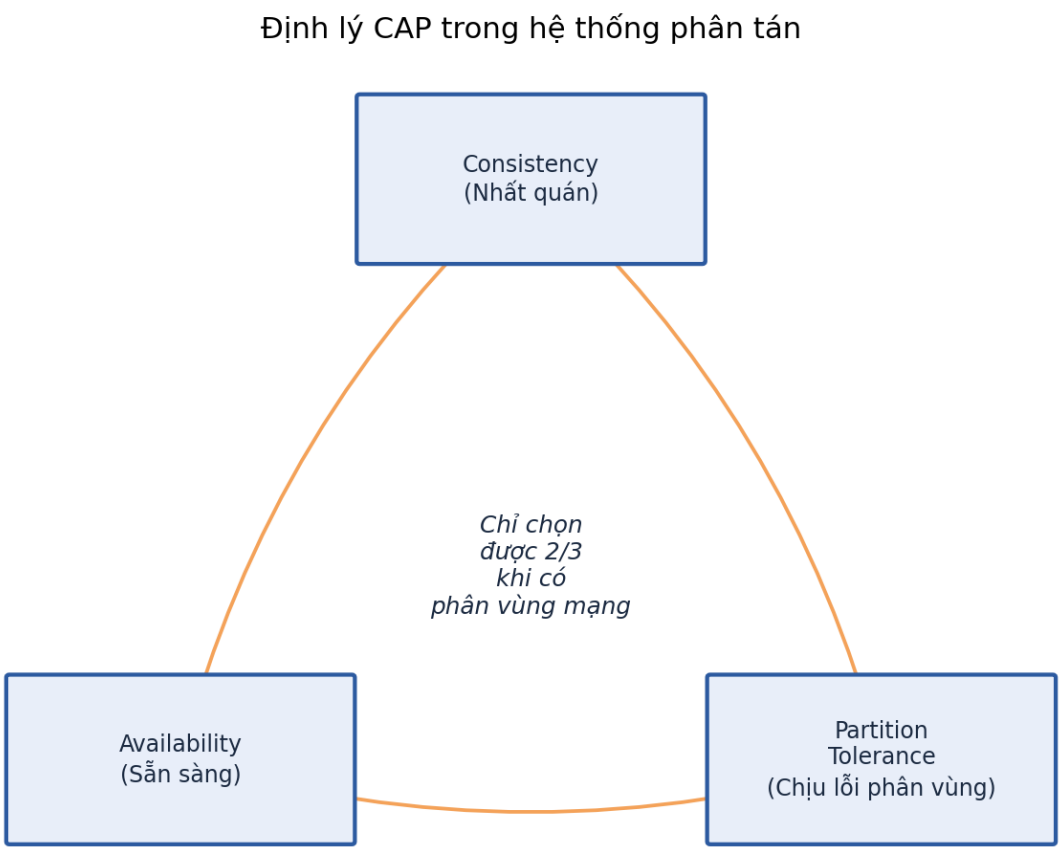
7.6. Thực hành Lựa chọn Kiến trúc

Phần cuối chương giúp sinh viên tổng hợp kiến thức đã học để đưa ra quyết định kiến trúc trong tình huống thực tế — một kỹ năng quan trọng đối với người làm hệ thống thông tin quản lý khi tham gia các dự án triển khai công nghệ tại doanh nghiệp.

7.6.1. Sự đánh đổi trong Kiến trúc: Định lý CAP

Khi thiết kế một hệ thống phân tán (dữ liệu được lưu trên nhiều node khác nhau, như trong kiến trúc vi dịch vụ), **định lý CAP** phát biểu rằng hệ thống chỉ có thể đảm bảo đồng thời **tối đa hai trong ba** thuộc tính sau khi xảy ra sự cố phân vùng mạng (network partition — các node mất kết nối với nhau):

- **Consistency (nhất quán)**: mọi node đều trả về dữ liệu mới nhất, giống nhau tại cùng một thời điểm.
- **Availability (sẵn sàng)**: mọi yêu cầu đều nhận được phản hồi, kể cả khi một số node gặp sự cố.
- **Partition Tolerance (chịu lỗi phân vùng)**: hệ thống vẫn hoạt động dù mạng giữa các node bị gián đoạn.



Hình 7.9 — Định lý CAP: chỉ chọn được 2 trong 3 thuộc tính khi xảy ra phân vùng mạng

Vì mạng phân tán thực tế luôn có khả năng xảy ra phân vùng, **Partition Tolerance** gần như là yêu cầu bắt buộc; bài toán thực sự là lựa chọn đánh đổi giữa **Consistency** và **Availability**. Ví dụ, hệ thống ngân hàng thường ưu tiên Consistency (số dư tài khoản phải chính xác tuyệt đối, thà tạm ngưng giao dịch còn hơn trả sai số dư), trong khi hệ thống mạng xã hội thường ưu tiên Availability (chấp nhận hiển thị số lượt thích cũ vài giây, miễn là hệ thống luôn phản hồi được).

7.6.2. Đánh giá và lựa chọn kiến trúc

Không có kiến trúc nào là "tốt nhất" cho mọi bài toán — việc lựa chọn phụ thuộc vào quy mô, ngân sách, năng lực đội ngũ và tốc độ tăng trưởng dự kiến của hệ thống.

Tình huống	Kiến trúc gợi ý	Lý do
Dự án khởi nghiệp, đội ngũ nhỏ, cần ra sản phẩm nhanh để kiểm chứng thị trường	Nguyên khối (Monolithic)	Phát triển và triển khai đơn giản, không tốn chi phí quản lý hạ tầng phân tán khi số lượng người dùng còn ít

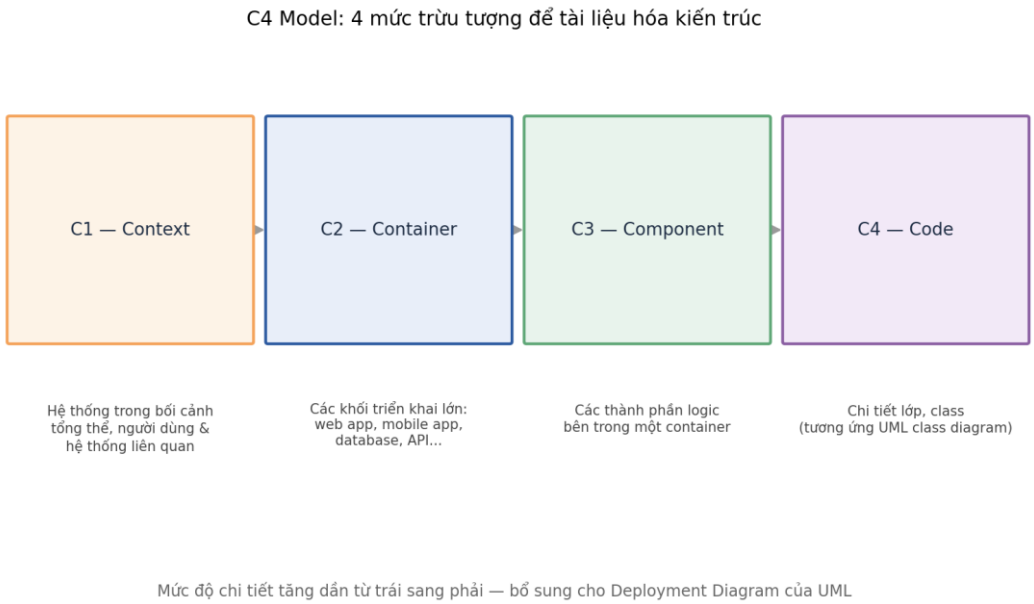
Tình huống	Kiến trúc gợi ý	Lý do
Hệ thống doanh nghiệp lớn, nhiều nhóm phát triển song song, cần mở rộng độc lập từng phần	Vi dịch vụ (Microservices)	Cho phép các nhóm triển khai độc lập, mở rộng đúng phần cần thiết, phù hợp năng lực đội ngũ và ngân sách vận hành lớn
Tác vụ xử lý không thường xuyên, tải không ổn định (ví dụ xử lý ảnh khi người dùng tải lên)	Phi máy chủ (Serverless)	Tránh trả phí cho tài nguyên nhàn rỗi, tự động mở rộng theo đúng lượng yêu cầu thực tế

Một chiến lược phổ biến trong thực tế là bắt đầu với kiến trúc nguyên khối để nhanh chóng đưa sản phẩm ra thị trường, sau đó **tái cấu trúc dần (refactor)** sang vi dịch vụ khi hệ thống đã chứng minh được giá trị và cần mở rộng quy mô — tránh đầu tư quá sớm vào một kiến trúc phức tạp mà chưa thực sự cần thiết.

7.6.3. Tài liệu hóa kiến trúc: C4 Model

Deployment diagram trong UML mô tả được các node vật lý và giao tiếp giữa chúng, nhưng không thể hiện đủ các mức chi tiết khác nhau mà các đối tượng liên quan (ban lãnh đạo, kiến trúc sư, lập trình viên) cần xem. **C4 Model** bổ sung cho khoảng trống này bằng bốn mức trừu tượng, từ tổng quát đến chi tiết:

- **C1 — Context**: hệ thống trong bối cảnh tổng thể, gồm người dùng và các hệ thống bên ngoài có tương tác.
- **C2 — Container**: các khối triển khai lớn bên trong hệ thống — ứng dụng web, ứng dụng di động, cơ sở dữ liệu, API.
- **C3 — Component**: các thành phần logic bên trong một container cụ thể.
- **C4 — Code**: mức chi tiết lớp, tương ứng với class diagram đã học ở các chương trước.



Hình 7.10 — Bốn mức trừu tượng của C4 Model

C4 Model giúp mỗi đối tượng liên quan chỉ cần xem đúng mức chi tiết phù hợp với vai trò của mình — ban lãnh đạo xem C1 để hiểu hệ thống phục vụ ai, kiến trúc sư xem C2 để quyết định hạ tầng, lập trình viên xem C3–C4 để hiện thực chi tiết — thay vì phải diễn giải toàn bộ thông tin trên một biểu đồ UML duy nhất.

Tóm tắt chương

- Kiến trúc phần mềm mô tả cấu trúc tổng thể của hệ thống — các thành phần chính, vị trí triển khai và cách chúng giao tiếp — khác với thiết kế chi tiết vốn tập trung vào bên trong từng thành phần.
- Các kiến trúc truyền thống (Client-Server, Nguyên khối, Phân tầng/MVC) là nền tảng, phù hợp với hệ thống quy mô vừa và nhỏ.
- Các kiến trúc chuyển tiếp (SOA, Event-Driven) mở ra khả năng kết nối nhiều hệ thống độc lập với nhau.
- Các kiến trúc hiện đại (Microservices, Cloud-Native, Serverless) đáp ứng nhu cầu mở rộng quy mô lớn và tốc độ phát triển sản phẩm nhanh của doanh nghiệp công nghệ ngày nay.
- Việc lựa chọn kiến trúc luôn là bài toán đánh đổi giữa các thuộc tính chất lượng (khả năng mở rộng, độ tin cậy, bảo mật, hiệu năng), không tồn tại kiến trúc "tốt nhất" cho mọi tình huống.
- C4 Model là công cụ tài liệu hóa kiến trúc theo nhiều mức trừu tượng, bổ sung cho Deployment Diagram của UML.

Câu hỏi ôn tập và bài tập

- Phân biệt kiến trúc phần mềm và thiết kế phần mềm chi tiết bằng ví dụ cụ thể của một hệ thống bạn đã học ở chương trước.
- So sánh ưu, nhược điểm của kiến trúc nguyên khối và kiến trúc vi dịch vụ. Trong trường hợp nào nên chọn kiến trúc nào?
- Giải thích vì sao nhiều doanh nghiệp chuyển từ SOA có ESB sang kiến trúc vi dịch vụ.
- Cho một hệ thống đặt vé xem phim trực tuyến với lượng truy cập tăng đột biến vào các dịp phim mới ra mắt: đề xuất kiến trúc phù hợp và giải thích lựa chọn dựa trên các thuộc tính chất lượng đã học.
- Áp dụng định lý CAP để phân tích: hệ thống quản lý kho hàng của một chuỗi siêu thị nên ưu tiên Consistency hay Availability khi xảy ra sự cố mạng giữa các chi nhánh? Giải thích.
- Vẽ mô hình C4 (mức Context và Container) cho một hệ thống thương mại điện tử đơn giản mà bạn tự đề xuất.